

SHETH LUJ AND SIR MV COLLEGE
Scala for Data Science M2 - 3 , 4, 5 and 6

Aim:

3. Write a program to compute frequency distribution and cumulative frequency of a dataset.

Code:

```
import scala.io.Source
```

```
object S091_Shreyash_M2_P3 {  
  def main(args: Array[String]): Unit = {  
  
    println("Shreyash Kadam S091")  
  
    val stream = getClass.getResourceAsStream("/all_stocks_5yr.csv")  
  
    if (stream == null) {  
      println("Error: File not found in resources folder!")  
      return  
    }  
  
    val file = Source.fromInputStream(stream)  
  
    val prices = file.getLines().drop(1).flatMap { line =>  
      val cols = line.split(",")  
  
      if (cols.length >= 7 && cols(6).trim == "AAL")  
        cols(4).trim.toDoubleOption  
      else  
        None  
    }.toList  
  
    file.close()  
  
    val minPrice = prices.min  
    val maxPrice = prices.max  
  
    val classSize = 10.0  
  
    val start = math.floor(minPrice / classSize) * classSize  
    val end = math.ceil(maxPrice / classSize) * classSize  
  
    var cumulative = 0  
  
    println("\nFrequency Distribution")  
    println("Price Range\tFrequency\tCumulative")
```

SHETH LUJ AND SIR MV COLLEGE
Scala for Data Science M2 - 3 , 4, 5 and 6

```
var lower = start

while (lower < end) {

  val upper = lower + classSize

  val frequency = prices.count { price =>
    price >= lower && price < upper
  }

  cumulative = cumulative + frequency

  println(f"$lower%.2f - $upper%.2f\t$frequency%\td\t$cumulative%d")

  lower = upper
}
println(s"Total Records: ${prices.length}")
}
```

Output:

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Shreyash Kadam S091

Frequency Distribution
Price Range Frequency    Cumulative
10.00 - 20.00    164      164
20.00 - 30.00    87       251
30.00 - 40.00   332      583
40.00 - 50.00   537     1120
50.00 - 60.00   139     1259
Total Records: 1259

Process finished with exit code 0
```

SHETH LUJ AND SIR MV COLLEGE
Scala for Data Science M2 - 3 , 4, 5 and 6

Aim:

4. Sort a dataset by a specific column and extract the top 5 rows.

Code:

```
import scala.io.Source

object S091_Shreyash_M2_P4 {
  def main(args: Array[String]): Unit = {

    println("Shreyash Kadam S091")

    val stream = getClass.getResourceAsStream("/all_stocks_5yr.csv")

    if (stream == null) {
      println("Error: File not found in resources folder!")
      return
    }

    val file = Source.fromInputStream(stream)

    val data = file.getLines().drop(1).flatMap { line =>
      val cols = line.split(",")

      if (cols.length >= 7 && cols(6).trim == "AAL") {
        for {
          close <- cols(4).trim.toDoubleOption
        } yield (cols(0), close)
      } else {
        None
      }
    }.toList

    file.close()

    val sortedData = data.sortBy(_._2).reverse

    val top5 = sortedData.take(5)

    println("\nTop 5 AAL Closing Prices:")
    top5.foreach { row =>
      println(s>Date: ${row._1} Close: ${row._2}")
    }
  }
}
```

Shreyash Kadam
S091

```
}  
}
```

Output:

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...  
Shreyash Kadam S091  
  
Top 5 AAL Closing Prices:  
Date: 2018-01-12   Close: 58.47  
Date: 2018-01-18   Close: 58.34  
Date: 2018-01-23   Close: 58.29  
Date: 2018-01-17   Close: 58.16  
Date: 2018-01-22   Close: 58.1  
  
Process finished with exit code 0
```

SHETH LUJ AND SIR MV COLLEGE
Scala for Data Science M2 - 3 , 4, 5 and 6

Aim:

5. Implement linear regression using Breeze. Fit a model to a small dataset and predict a value.

Code:

```
import breeze.linalg._
import scala.io.Source

object S091_Shreyash_M2_P5 {
  def main(args: Array[String]): Unit = {

    println("Shreyash Kadam S091")

    val stream = getClass.getResourceAsStream("/all_stocks_5yr.csv")

    if (stream == null) {
      println("Error: File not found in resources folder!")
      return
    }

    val file = Source.fromInputStream(stream)

    val data = file.getLines().drop(1).flatMap { line =>
      val cols = line.split(",")

      if (cols.length >= 7 && cols(6).trim == "AAL") {
        for {
          x <- cols(1).trim.toDoubleOption
          y <- cols(4).trim.toDoubleOption
        } yield (x, y)
      } else {
        None
      }
    }.take(50).toList

    file.close()

    val x = DenseVector(data.map(_._1).toArray)
    val y = DenseVector(data.map(_._2).toArray)

    val X = DenseMatrix.horzcat(
      DenseVector.ones[Double](x.length).toDenseMatrix.t,
      x.toDenseMatrix.t
    )
```

SHETH LUJ AND SIR MV COLLEGE
Scala for Data Science M2 - 3 , 4, 5 and 6

```
val coefficients =  
  inv(X.t * X) * X.t * y  
  
val intercept = coefficients(0)  
val slope = coefficients(1)  
  
println("\nLinear Regression Model:")  
println(f"Close = intercept%.4f + (slope%.4f * Open)")  
  
val openPrice = 40.0  
  
val predictedClose =  
  intercept + slope * openPrice  
  
println(f"\nOpen Price: $openPrice%.2f")  
println(f"\nPredicted Close Price: $predictedClose%.2f")  
  
println(s"\nDataset Size: ${data.length} records")  
}  
}
```

Output:

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...  
Shreyash Kadam S091  
Aug 22, 2026 11:53:24 PM dev.ludovic.netlib.blas.InstanceBuilder initializeNative  
WARNING: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS  
trying to return java 16 instance  
Aug 22, 2026 11:53:24 PM dev.ludovic.netlib.blas.InstanceBuilder initializeJava  
WARNING: Failed to load implementation from:dev.ludovic.netlib.blas.VectorBLAS  
return java 11 instance  
Aug 22, 2026 11:53:24 PM dev.ludovic.netlib.lapack.InstanceBuilder initializeNative  
WARNING: Failed to load implementation from:dev.ludovic.netlib.lapack.JNILAPACK  
  
Linear Regression Model:  
Close = 0.7221 + (0.9506 * Open)  
  
Open Price: 40.00  
Predicted Close Price: 38.75  
  
Dataset Size: 50 records  
  
Process finished with exit code 0
```

SHETH LUJ AND SIR MV COLLEGE
Scala for Data Science M2 - 3 , 4, 5 and 6

Aim:

6. Perform logistic regression using Breeze. Classify a dataset with binary labels.

Code:

```
import breeze.linalg._
import scala.io.Source

object S091_Shreyash_M2_P6 {
  def main(args: Array[String]): Unit = {

    println("Shreyash Kadam S091")

    val stream = getClass.getResourceAsStream("/all_stocks_5yr.csv")

    if (stream == null) {
      println("Error: File not found in resources folder!")
      return
    }

    val file = Source.fromInputStream(stream)

    val data = file.getLines().drop(1).flatMap { line =>
      val cols = line.split(",")

      if (cols.length >= 7 && cols(6).trim == "AAL") {
        for {
          open <- cols(1).trim.toDoubleOption
          close <- cols(4).trim.toDoubleOption
        } yield {
          val label = if (close > open) 1.0 else 0.0
          (open, label)
        }
      } else {
        None
      }
    }.take(50).toList

    file.close()

    val x = DenseVector(data.map(_._1).toArray)
    val y = DenseVector(data.map(_._2).toArray)

    var beta0 = 0.0
    var beta1 = 0.0
```

Shreyash Kadam
S091

SHETH LUJ AND SIR MV COLLEGE
Scala for Data Science M2 - 3 , 4, 5 and 6

```
val learningRate = 0.0001
val iterations = 10000

for (i <- 0 until iterations) {

  var sum0 = 0.0
  var sum1 = 0.0

  for (j <- 0 until x.length) {

    val z = beta0 + beta1 * x(j)
    val probability = 1.0 / (1.0 + math.exp(-z))

    sum0 = sum0 + (probability - y(j))
    sum1 = sum1 + (probability - y(j)) * x(j)
  }

  beta0 = beta0 - learningRate * sum0
  beta1 = beta1 - learningRate * sum1
}

println("\nLogistic Regression Model:")
println(f"Probability = 1 / (1 + e^(-($beta0%.4f + $beta1%.4f * Open))))")

val openPrice = 40.0

val z = beta0 + beta1 * openPrice

val probability =
  1.0 / (1.0 + math.exp(-z))

val prediction =
  if (probability >= 0.5) 1 else 0

println(f"\nOpen Price: $openPrice%.2f")
println(f"Probability of Price Increase: $probability%.4f")

if (prediction == 1)
  println("Prediction: Price Increase")
else
  println("Prediction: No Price Increase")

println(s"\nDataset Size: ${data.length} records")
```



```
}  
}
```

Output:

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...  
Shreyash Kadam S091  
  
Logistic Regression Model:  
Probability = 1 / (1 + e^(-(0.3321 + -0.0233 * Open)))  
  
Open Price: 40.00  
Probability of Price Increase: 0.3541  
Prediction: No Price Increase  
  
Dataset Size: 50 records  
  
Process finished with exit code 0
```